

JTAG Debugging

op de ESP32-S3

in de Arduino-omgeving

Stap-voor-stap handleiding voor hardware-debugging

Versie 1.0 • 2026

1. Inleiding

JTAG (Joint Test Action Group) is een internationale standaard (IEEE 1149.1) voor het testen en debuggen van elektronische schakelingen. Op de ESP32-S3 biedt JTAG directe toegang tot de processor via hardware, waardoor u breakpoints kunt instellen, registers kunt inspecteren en geheugen kunt lezen — allemaal via de usb kabel zonder extra hardware.

Dit document beschrijft hoe u JTAG instelt en gebruikt in combinatie met de Arduino IDE op de ESP32-S3.

1.1 Waarom JTAG in plaats van `Serial.print()`?

- Real-time inspectie van variabelen en registers zonder code te wijzigen
- Hardware-breakpoints die stoppen op een exacte instructie
- Stack-trace bekijken bij crashes
- Geheugeninhoud (RAM, Flash) direct uitlezen en schrijven
- Geschikt voor tijdkritische en interrupt-gedreven code

2. De JTAG-interface van de ESP32-S3

2.1 Ingebouwde USB-JTAG

De ESP32-S3 beschikt over een ingebouwde USB-JTAG-interface via de USB. Dit betekent dat u geen extra hardware nodig heeft om real-time te debuggen.

Voorwaarde: het board moet zijn aangesloten op de USB-poort die direct verbonden is met de ESP32-S3 zelf (vaak aangeduid als **USB** of **UART/USB** op het board).

Als u ook Serial wilt gebruiken sluit dan een extra USB naar TTL kabel aan op pinnen TX en RX. Elk `Serial.println()` statement zal nu uitvoer produceren op die pinnen. Het is dus niet mogelijk om JTAG te gebruiken en tegelijk seriële data op de USB poort te printen !

3. Benodigde software

3.1 Arduino IDE met ESP32-ondersteuning

- Arduino IDE 2.x (aanbevolen — heeft ingebouwde debugger-interface)
- ESP32 Arduino-kernpakket versie 2.0.5 of hoger (via Boards Manager)

Voeg de volgende URL toe aan Bestand > Voorkeuren > veld 'Extra Boards Manager-URLs':

```
https://espressif.github.io/arduino-esp32/package\_esp32\_index.json
```

3.4 USB-stuurprogramma (Windows)

Op Windows is het Zadig-hulpprogramma nodig om het juiste WinUSB-stuurprogramma te installeren voor de ingebouwde USB-JTAG-interface:

- Download Zadig via <https://zadig.akeo.ie/>
- Sluit de ESP32-S3 aan via USB
- Selecteer in Zadig: Options > List All Devices
- Kies 'ESP32-S3 (Interface 2)'
- Installeer WinUSB als stuurprogramma

4. Configuratie in Arduino IDE 2

4.1 Juiste board en poort selecteren

- Kies Tools > Board > ESP32 > ESP32S3 Dev Module
- Kies de 'Upload Speed' van 921600 baud.
- Zet de 'USB Mode' op 'Hardware CDC and JTAG'
- Kies de juiste COM-poort (of /dev/ttyACM0 op Linux)

4.2 Debug-level instellen

Stel het debug-niveau in via Tools > Core Debug Level > Debug. Dit zorgt voor extra output via de seriële monitor en behoudt debugsymbolen in de firmware.

5. Debuggen stap voor stap

5.1 Sketch compileren en uploaden

Compileer en upload (pijl icoon linksboven) de sketch via de normale Arduino IDE-workflow (Ctrl+U). Zorg dat de firmware naar de ESP32-S3 is geschreven voordat u de debugger start.

Kies vervolgens het debug symbol rechts naast de pijl om te gaan debuggen.

De volgende symbooltjes verschijnen waarmee u nu door het lopende programma kunt gaan lopen.



Van links naar rechts:

- Met 'continue' kunt u een onderbroken programma weer verder laten lopen.
- Met 'Step over' wordt het volgende statement uitgevoerd zonder erin te duiken.
- Met 'Step into' wordt het volgende statement uitgevoerd door erin te duiken.
- De 'Step uit' is om een niveau hoger in het programma te gaan.
- Met de 'Restart' wordt het programma fris opnieuw gestart.
- De 'Stop' is er om een lopend programma te onderbreken.

5.2 Breakpoints instellen

Vanuit Arduino IDE 2 klikt u in de marge naast een regelnummer om een breakpoint in te stellen. Er verschijnt een rode stip op die regel. Als het programma nu bij het breakpoint komt dan stopt het.

5.2 Variabelen bekijken

Op het moment dat het programma tijdelijk gestop is zijn de variabelen te bekijken door boven de naam in de broncode met de muis te gaan hangen.

5.3 Call stack en Threads bekijken

Als het programma stil staat dan zie je links de 'Threads' staan. Arduino loopt op core ipc0. (In principe kan nog een ander programma tegelijk opgestart worden op de andere core ipc1)

De call stack (de stapel van aanroepen van begin van het programma tot waar we nu zijn) is te zien onder de Threads tab. Als je dubbelklikt op een element uit deze stap dan zie je waar vanuit de broncode de aanroep is geweest.

6. Tips en aanbevelingen

6.1 Compileer altijd met debugsymbolen

Stel in de Arduino IDE Tools > Optimize in op 'Debug (-Og)'. Dit behoudt symbool informatie en uitlijning van broncode met machinetaal, wat breakpoints betrouwbaarder maakt.

7. Referenties en verdere documentatie

- Espressif JTAG Debugging Guide: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-guides/jtag-debugging/>
 - Arduino ESP32 GitHub-repository: <https://github.com/espressif/arduino-esp32>
 - GDB-handleiding (GNU): <https://sourceware.org/gdb/current/onlinedocs/gdb/>
 - Zadig USB-stuurprogrammatool: <https://zadig.akeo.ie/>
-